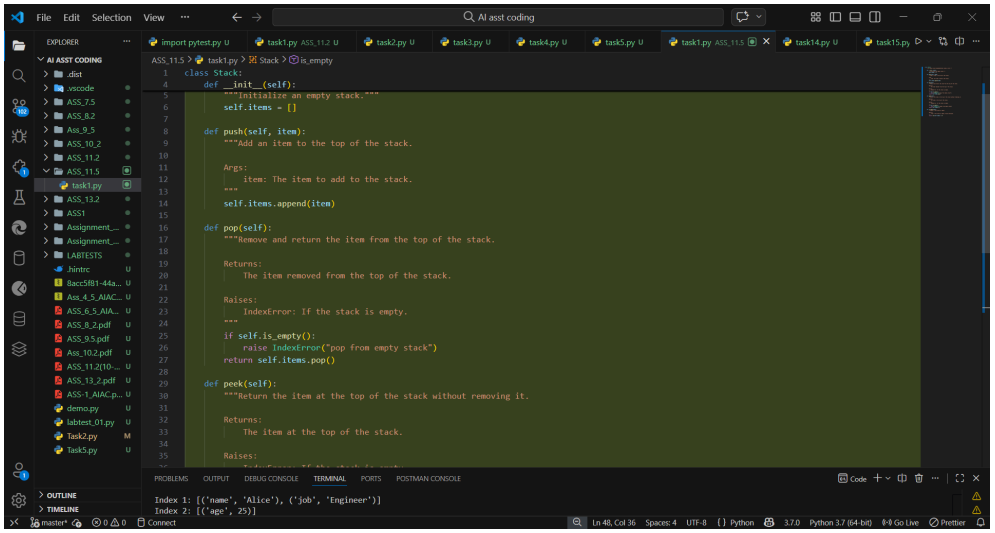
ASSIGNMENT 11.5
AI ASSISTED CODING

E.SAI ANURATH | BATCH-29 | 2303A51560 | DATE:10-03-2026

> Task Description #1 – Stack Implementation
> Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.
> Sample Input Code:
> class Stack:
> pass
> Expected Output:
> • A functional stack implementation with all required methods and docstrings.

```
class Stack:
    """A simple stack implementation using a list."""

    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack.

        Args:
            item: The item to add to the stack.
        """
        self.items.append(item)

    def pop(self):
        """Remove and return the item from the top of the stack.

        Returns:
            The item removed from the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("pop from empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it.

        Returns:
            The item at the top of the stack.

        Raises:
            IndexError: If the stack is empty.
        """
```

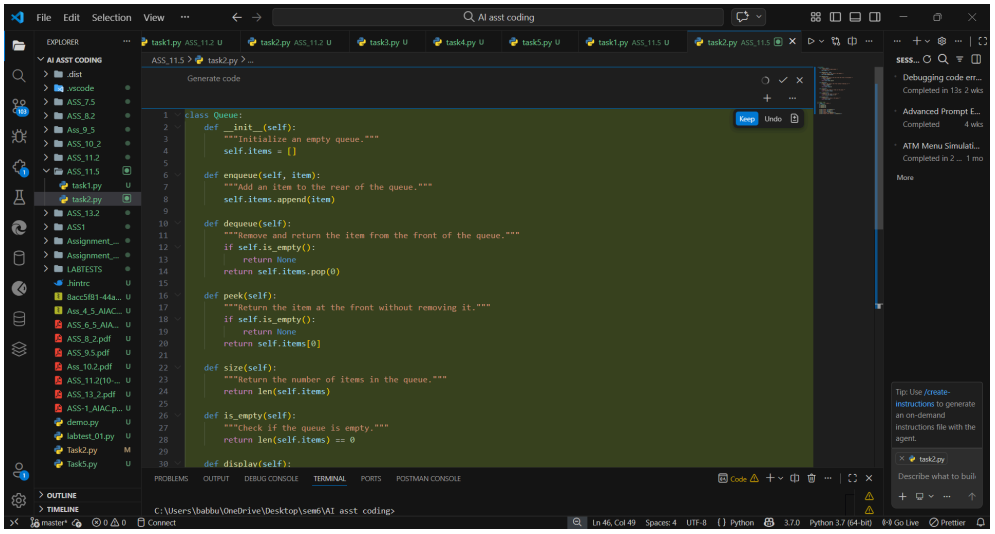
```
        if self.is_empty():
            raise IndexError("peek from empty stack")
        return self.items[-1]

    def is_empty(self):
        """Check if the stack is empty.

        Returns:
            True if the stack is empty, False otherwise.
        """
        return len(self.items) == 0
```

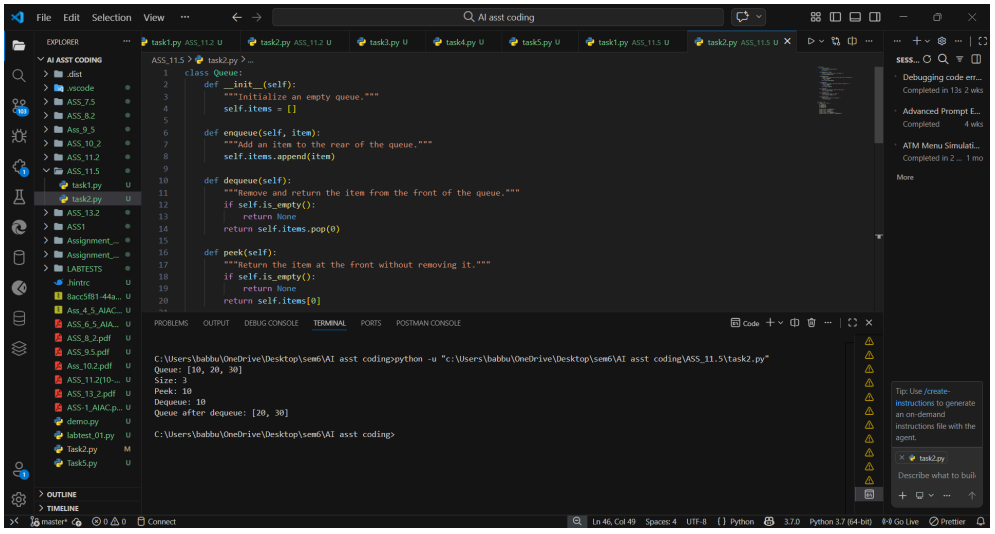


> Task Description #2 – Queue Implementation
> Task: Use AI to implement a Queue using Python lists.
> Sample Input Code:
> class Queue:
> pass
> Expected Output:
> • FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```
class Queue:
    def __init__(self):
        """Initialize an empty queue."""
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item from the front of the queue."""
        if self.is_empty():
            return None
        return self.items.pop(0)
```

```
    def peek(self):
        """Return the item at the front without removing it."""
        if self.is_empty():
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0

    def display(self):
        """Display all items in the queue."""
        return self.items


# Example usage:
if __name__ == "__main__":
    q = Queue()
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)

    print(f"Queue: {q.display()}")
    print(f"Size: {q.size()}")
    print(f"Peek: {q.peek()}")
    print(f"Dequeue: {q.dequeue()}")
    print(f"Queue after dequeue: {q.display()}")
```

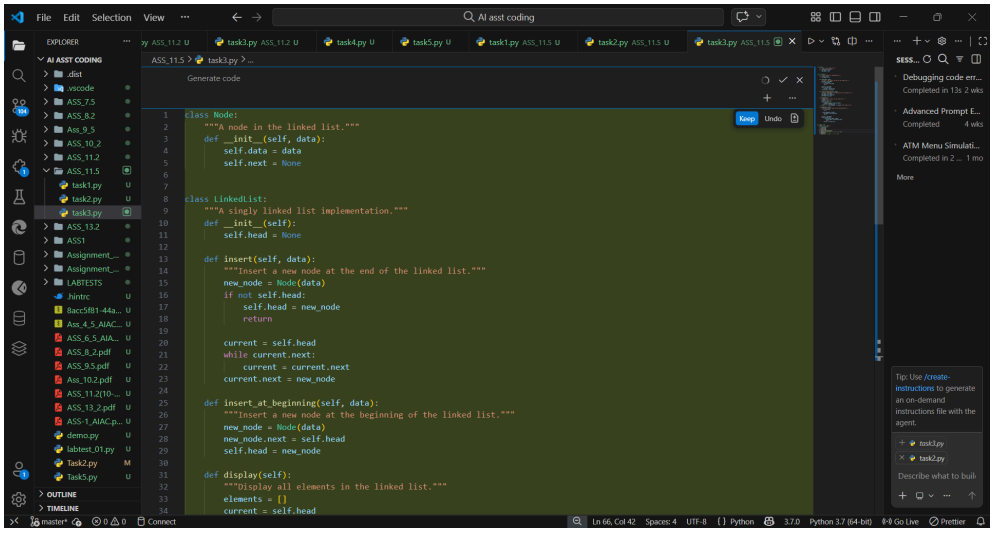


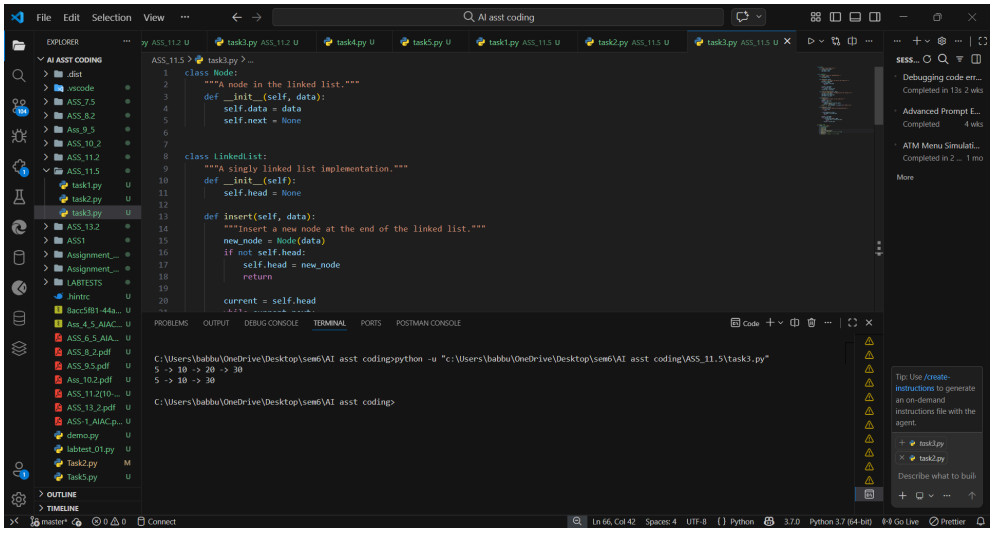


> Task Description #3 – Linked List
> Task: Use AI to generate a Singly Linked List with insert and display methods.
> Sample Input Code:
> class Node:
> pass
> class LinkedList:
> pass
> Expected Output:
> • A working linked list implementation with clear method documentation.

```
class Node:
    """A node in the linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    """A singly linked list implementation."""
    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node at the end of the linked list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

    def insert_at_beginning(self, data):
        """Insert a new node at the beginning of the linked list."""
```

```
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def display(self):
        """Display all elements in the linked list."""
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) if elements else "Empty List")

    def delete(self, data):
        """Delete the first node with the specified data."""
        if not self.head:
            return

        if self.head.data == data:
            self.head = self.head.next
            return

        current = self.head
        while current.next:
            if current.next.data == data:
                current.next = current.next.next
                return
            current = current.next


# Example usage
if __name__ == "__main__":
    ll = LinkedList()
    ll.insert(10)
    ll.insert(20)
    ll.insert(30)
    ll.insert_at_beginning(5)
    ll.display()  # Output: 5 -> 10 -> 20 -> 30
    ll.delete(20)
    ll.display()  # Output: 5 -> 10 -> 30
```



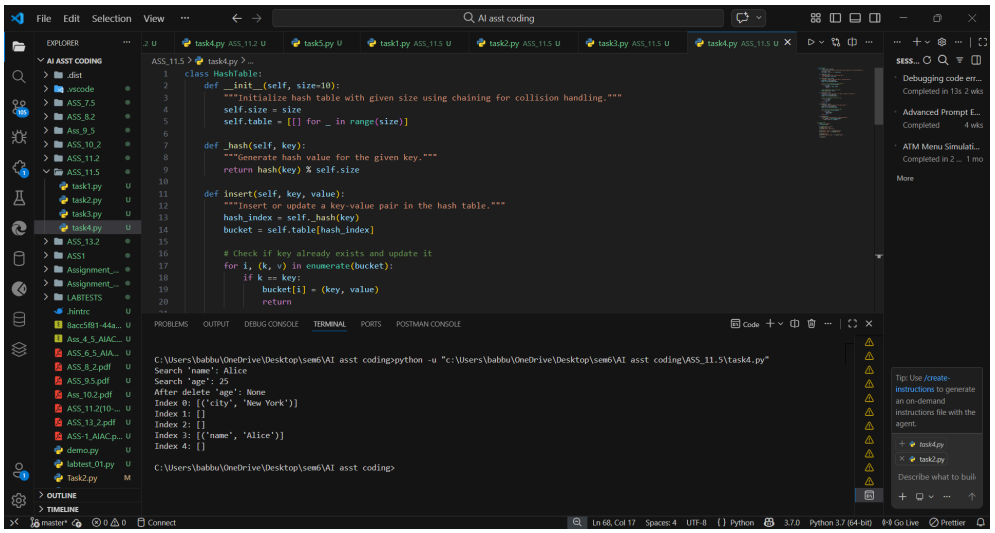


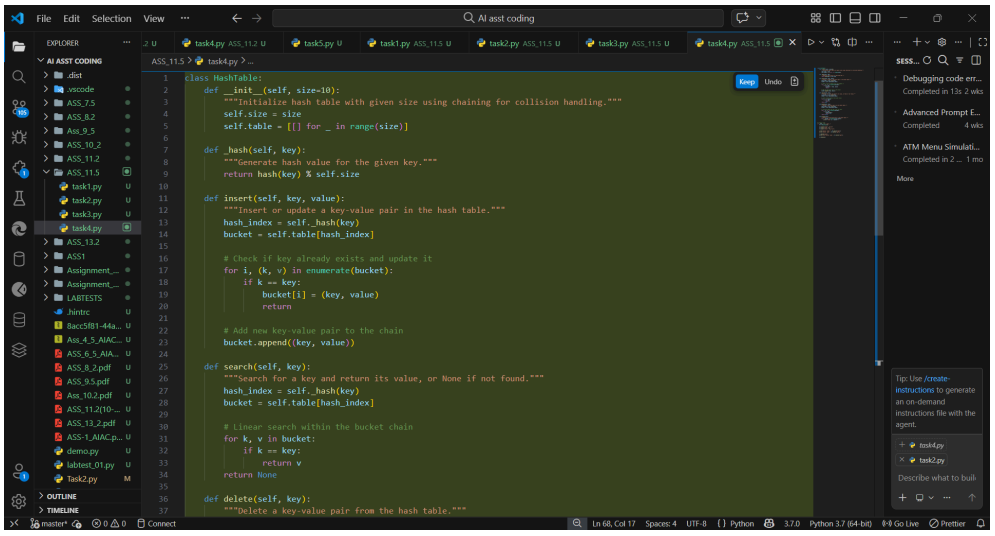
> Task Description #4 – Hash Table
> Task: Use AI to implement a hash table with basic insert, search, and delete methods.
> Sample Input Code:
> class HashTable:
> pass
> Expected Output:
> • Collision handling using chaining, with well-commented methods.

```
class HashTable:
    def __init__(self, size=10):
        """Initialize hash table with given size using chaining for collision handlin
g."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate hash value for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert or update a key-value pair in the hash table."""
```

```
        hash_index = self._hash(key)
        bucket = self.table[hash_index]

        # Check if key already exists and update it
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return

        # Add new key-value pair to the chain
        bucket.append((key, value))

    def search(self, key):
        """Search for a key and return its value, or None if not found."""
        hash_index = self._hash(key)
        bucket = self.table[hash_index]

        # Linear search within the bucket chain
        for k, v in bucket:
            if k == key:
                return v
        return None

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        hash_index = self._hash(key)
        bucket = self.table[hash_index]

        # Find and remove the key from the chain
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                return True
        return False

    def display(self):
        """Display all key-value pairs in the hash table."""
        for i, bucket in enumerate(self.table):
            print(f"Index {i}: {bucket}")


# Example usage
if __name__ == "__main__":
    ht = HashTable(5)

    ht.insert("name", "Alice")
    ht.insert("age", 25)
    ht.insert("city", "New York")

    print("Search 'name':", ht.search("name"))
    print("Search 'age':", ht.search("age"))

    ht.delete("age")
    print("After delete 'age':", ht.search("age"))
```

```
    ht.display()
```

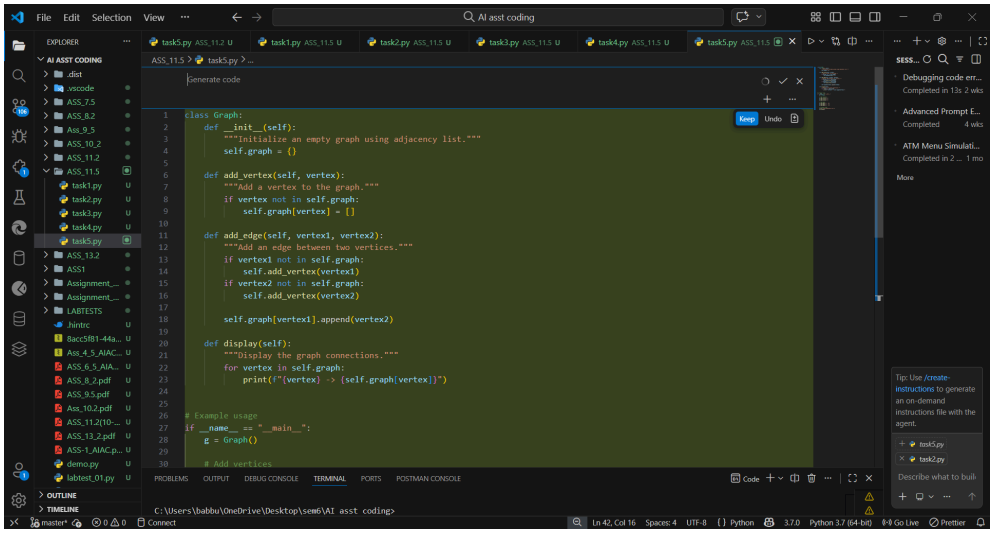


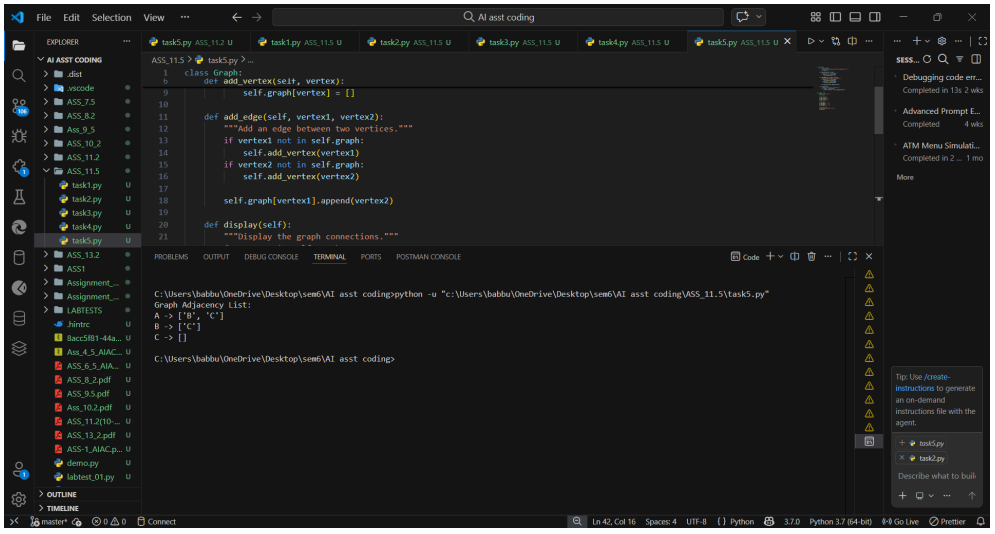


> Task Description #5 – Graph Representation
> Task: Use AI to implement a graph using an adjacency list.
> Sample Input Code:
> class Graph:
> pass
> Expected Output:
> • Graph with methods to add vertices, add edges, and display connections.

```
class Graph:
    def __init__(self):
        """Initialize an empty graph using adjacency list."""
        self.graph = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []
```

```
    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices."""
        if vertex1 not in self.graph:
            self.add_vertex(vertex1)
        if vertex2 not in self.graph:
            self.add_vertex(vertex2)

        self.graph[vertex1].append(vertex2)

    def display(self):
        """Display the graph connections."""
        for vertex in self.graph:
            print(f"{vertex} -> {self.graph[vertex]}")


# Example usage
if __name__ == "__main__":
    g = Graph()

    # Add vertices
    g.add_vertex('A')
    g.add_vertex('B')
    g.add_vertex('C')

    # Add edges
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'C')

    # Display graph
    print("Graph Adjacency List:")
    g.display()
```





> Task Description #6: Smart Hospital Management System – Data Structure Selection
> A hospital wants to develop a Smart Hospital Management System that handles:
>
> 1. Patient Check-In System – Patients are registered and treated in order of arrival.
> 2. Emergency Case Handling – Critical patients must be treated first.
> 3. Medical Records Storage – Fast retrieval of patient details using ID.
> 4. Doctor Appointment Scheduling – Appointments sorted by time.
> 5. Hospital Room Navigation – Represent connections between wards and rooms.
>
> Student Task
> • For each feature, select the most appropriate data structure from the list below:
> o Stack
> o Queue
> o Priority Queue
> o Linked List
> o Binary Search Tree (BST)
> o Graph
> o Hash Table
> o Deque
> • Justify your choice in 2–3 sentences per feature.
> • Implement one selected feature as a working Python program with AI-assisted code generation.
> Expected Output:
> • A table mapping feature → chosen data structure → justification.
> • A functional Python program implementing the chosen feature with comments and docstrings.

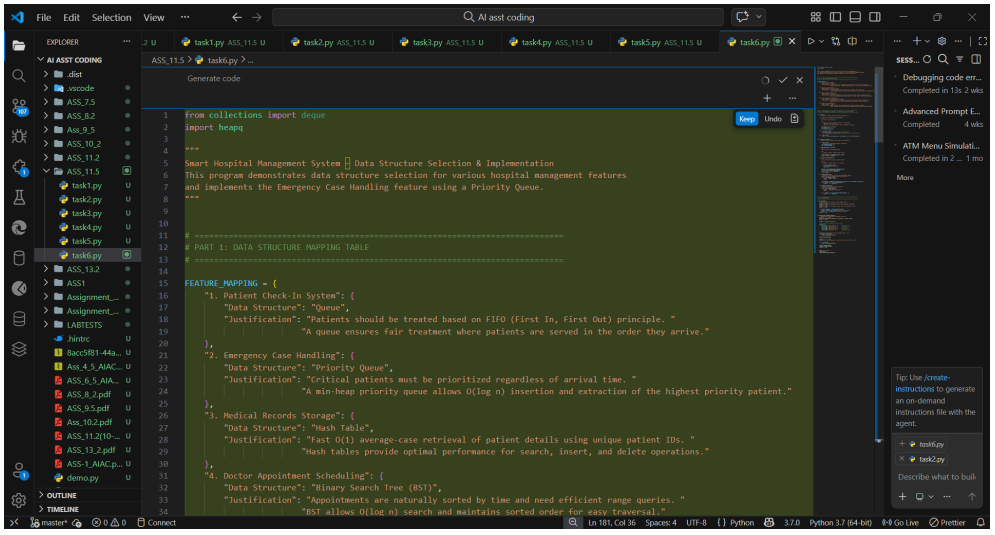
```
from collections import deque
import heapq

"""
Smart Hospital Management System — Data Structure Selection & Implementation
```

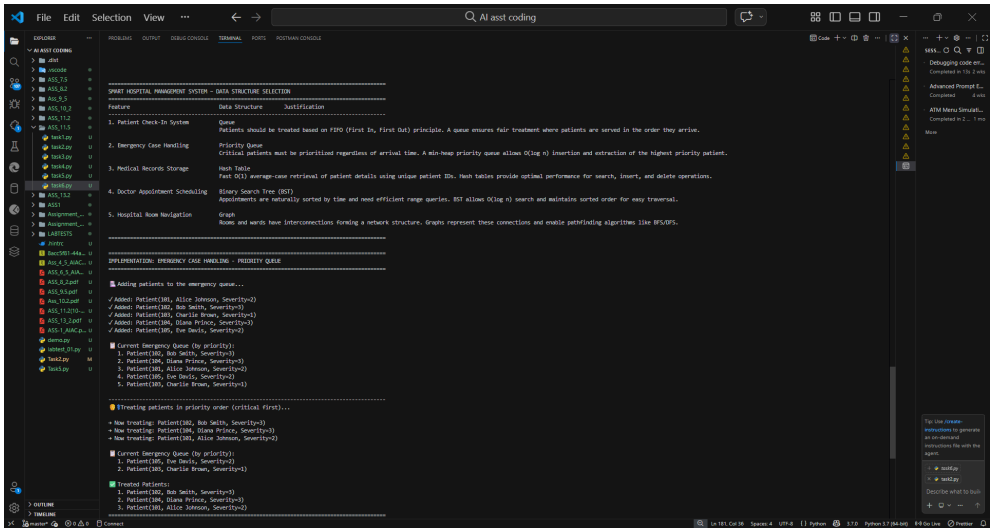
```
This program demonstrates data structure selection for various hospital management fea
tures
and implements the Emergency Case Handling feature using a Priority Queue.
"""


# ==============================================================================
# PART 1: DATA STRUCTURE MAPPING TABLE
# ==============================================================================

FEATURE_MAPPING = {
    "1. Patient Check-In System": {
        "Data Structure": "Queue",
        "Justification": "Patients should be treated based on FIFO (First In, First Ou
t) principle. "
                         "A queue ensures fair treatment where patients are served in t
he order they arrive."
    },
    "2. Emergency Case Handling": {
        "Data Structure": "Priority Queue",
        "Justification": "Critical patients must be prioritized regardless of arrival
time. "
                         "A min-heap priority queue allows O(log n) insertion and extra
ction of the highest priority patient."
    },
    "3. Medical Records Storage": {
        "Data Structure": "Hash Table",
        "Justification": "Fast O(1) average-case retrieval of patient details using un
ique patient IDs. "
                         "Hash tables provide optimal performance for search, insert, a
nd delete operations."
    },
    "4. Doctor Appointment Scheduling": {
        "Data Structure": "Binary Search Tree (BST)",
        "Justification": "Appointments are naturally sorted by time and need efficient
range queries. "
                         "BST allows O(log n) search and maintains sorted order for eas
y traversal."
    },
    "5. Hospital Room Navigation": {
        "Data Structure": "Graph",
        "Justification": "Rooms and wards have interconnections forming a network stru
cture. "
                         "Graphs represent these connections and enable pathfinding alg
orithms like BFS/DFS."
    }
}

# ==============================================================================
# PART 2: IMPLEMENTATION - Emergency Case Handling with Priority Queue
# ==============================================================================

class Patient:
    """Represents a patient with basic medical information."""
```

```
    def __init__(self, patient_id, name, severity):
        """
        Initialize a patient object.

        Args:
            patient_id (int): Unique identifier for the patient
            name (str): Patient's name
            severity (int): Severity level (1=low, 2=medium, 3=critical)
        """
        self.patient_id = patient_id
        self.name = name
        self.severity = severity

    def __lt__(self, other):
        """Compare patients by severity (for heap ordering)."""
        return self.severity > other.severity  # Higher severity = higher priority

    def __repr__(self):
        return f"Patient({self.patient_id}, {self.name}, Severity={self.severity})"


class EmergencyCaseHandler:
    """Manages emergency patient cases using a Priority Queue."""

    def __init__(self):
        """Initialize the emergency queue as a min-heap."""
        self.emergency_queue = []
        self.treated_patients = []

    def add_patient(self, patient):
        """
        Add a patient to the emergency queue.

        Args:
            patient (Patient): Patient object to add
        """
        heapq.heappush(self.emergency_queue, patient)
        print(f"✓ Added: {patient}")

    def treat_next_patient(self):
        """
        Remove and treat the highest priority patient.

        Returns:
            Patient: The next patient to be treated
        """
        if not self.emergency_queue:
            print("No patients in queue.")
            return None

        patient = heapq.heappop(self.emergency_queue)
        self.treated_patients.append(patient)
        print(f"→ Now treating: {patient}")
        return patient
```

```
    def display_queue(self):
        """Display all patients currently in the emergency queue."""
        print("\n📋 Current Emergency Queue (by priority):")
        if not self.emergency_queue:
            print("   Queue is empty.")
            return

        sorted_queue = sorted(self.emergency_queue, key=lambda p: p.severity, reverse=
True)
        for i, patient in enumerate(sorted_queue, 1):
            print(f"   {i}. {patient}")

    def display_treated(self):
        """Display all patients already treated."""
        print("\n✅ Treated Patients:")
        if not self.treated_patients:
            print("   No patients treated yet.")
            return

        for i, patient in enumerate(self.treated_patients, 1):
            print(f"   {i}. {patient}")


# ============================================================================
# PART 3: DEMONSTRATION
# ============================================================================

def print_table():
    """Print the feature-to-data-structure mapping table."""
    print("=" * 90)
    print("SMART HOSPITAL MANAGEMENT SYSTEM — DATA STRUCTURE SELECTION")
    print("=" * 90)
    print(f"{'Feature':<35} {'Data Structure':<20} {'Justification':<35}")
    print("-" * 90)

    for feature, details in FEATURE_MAPPING.items():
        print(f"{feature:<35} {details['Data Structure']:<20}")
        print(f"{'':35} {details['Justification']:<55}\n")
    print("=" * 90)


def demonstrate_emergency_handler():
    """Demonstrate the Emergency Case Handler system."""
    print("\n" + "=" * 90)
    print("IMPLEMENTATION: EMERGENCY CASE HANDLING - PRIORITY QUEUE")
    print("=" * 90)

    handler = EmergencyCaseHandler()

    # Add patients with different severity levels
    patients = [
        Patient(101, "Alice Johnson", 2),      # Medium severity
        Patient(102, "Bob Smith", 3),          # Critical
        Patient(103, "Charlie Brown", 1),      # Low severity
        Patient(104, "Diana Prince", 3),       # Critical
```

```
        Patient(105, "Eve Davis", 2),          # Medium severity
    ]

    print("\n🏥 Adding patients to the emergency queue...\n")
    for patient in patients:
        handler.add_patient(patient)

    handler.display_queue()

    print("\n" + "-" * 90)
    print("👨‍⚕️ Treating patients in priority order (critical first)...\n")

    for _ in range(3):
        handler.treat_next_patient()

    handler.display_queue()
    handler.display_treated()
    print("=" * 90)


if __name__ == "__main__":
    print_table()
    demonstrate_emergency_handler()
```




File Edit Selection View
AI asst coding
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
SMART HOSPITAL MANAGEMENT SYSTEM - DATA STRUCTURE SELECTION
Feature Data Structure Justification
1. Patient Check-In System Queue
Patients should be treated based on FIFO (First In, First Out) principle. A queue ensures fair treatment where patients are served in the order they arrive.
2. Emergency Case Handling Priority Queue
Critical patients must be prioritized regardless of arrival time. A min-heap priority queue allows O(log n) insertion and extraction of the highest priority patient.
3. Medical Records Storage Hash Table
Fast O(1) average-case retrieval of patient details using unique patient IDs. Hash tables provide optimal performance for search, insert, and delete operations.
4. Doctor Appointment Scheduling Binary Search Tree (BST)
Appointments are naturally sorted by time and need efficient range queries. BST allows O(log n) search and maintains sorted order for easy traversal.
5. Hospital Room Navigation Graph
Rooms and wards have interconnections forming a network structure. Graphs represent these connections and enable pathfinding algorithms like BFS/DFS.
IMPLEMENTATION: EMERGENCY CASE HANDLING - PRIORITY QUEUE
Adding patients to the emergency queue...
✓ Added: Patient(101, Alice Johnson, Severity=2)
✓ Added: Patient(102, Bob Smith, Severity=3)
✓ Added: Patient(103, Charlie Brown, Severity=1)
✓ Added: Patient(104, Diana Prince, Severity=3)
✓ Added: Patient(105, Eve Davis, Severity=2)
Current Emergency Queue (by priority):
1. Patient(102, Bob Smith, Severity=3)
2. Patient(104, Diana Prince, Severity=3)
3. Patient(101, Alice Johnson, Severity=2)
4. Patient(105, Eve Davis, Severity=2)
5. Patient(103, Charlie Brown, Severity=1)
Treating patients in priority order (critical first)...
• Now treating: Patient(102, Bob Smith, Severity=3)
• Now treating: Patient(104, Diana Prince, Severity=3)
• Now treating: Patient(101, Alice Johnson, Severity=2)
Current Emergency Queue (by priority):
1. Patient(105, Eve Davis, Severity=2)
2. Patient(103, Charlie Brown, Severity=1)
Treated Patients:
1. Patient(102, Bob Smith, Severity=3)
2. Patient(104, Diana Prince, Severity=3)
3. Patient(101, Alice Johnson, Severity=2)